

Extended E-R Features: Specialization, Generalization, and Aggregation

Visually Structured Notes with Diagrams

Note

Legend. Blue rounded rectangles: *entities*; orange diamonds: *relationships*; purple triangle: *ISA (specialization/generalization)*; yellow ellipses: *attributes*. Double edge = *total participation*.

Specialization (Top-Down)

Definition

Specialization creates lower-level (subclass) entity sets from a higher-level (superclass) entity set by dividing entities into meaningful subgroups with additional attributes or relationships. Depicted by an **ISA** triangle.

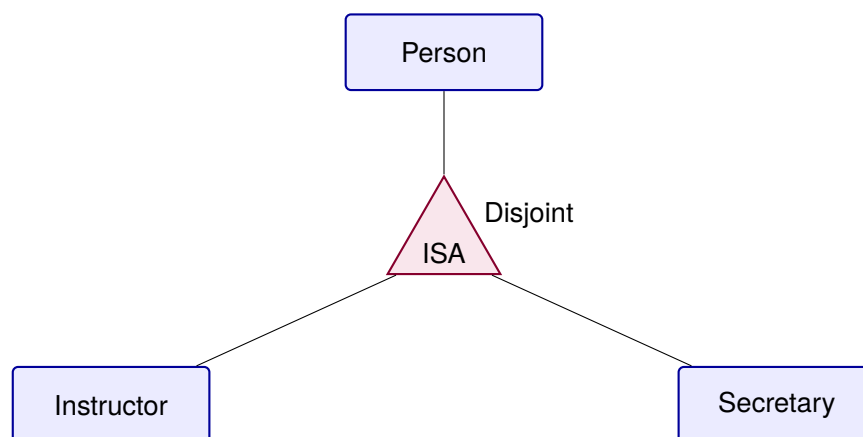
Attribute inheritance: subclasses inherit all attributes and relationship participation of their superclass.

Example

Overlapping example: employee and student.

Disjoint example: instructor and secretary.

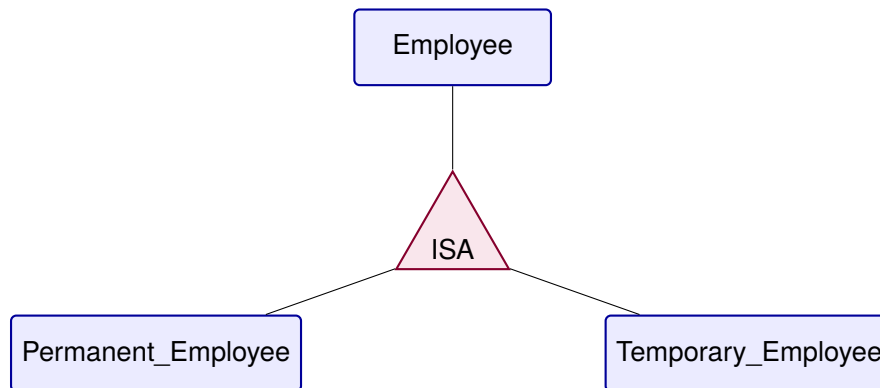
Completeness: total vs partial specialization.



Generalization (Bottom-Up)

Definition

Generalization combines multiple lower-level entity sets with common features into a higher-level entity set. It is the inverse of specialization; both are drawn with an ISA triangle.



Note

Multiple specializations can co-exist on the same superclass (e.g., Employee specialized by {Permanent, Temporary} and also by {Instructor, Secretary}). The ISA relationship is also called a *superclass–subclass* relationship.

Design Constraints on Specialization/Generalization

Type 1: Membership (How subclasses are formed)

- **Condition-defined (attribute-defined):** membership depends on a predicate or attribute condition.
Examples: Saving_Account vs Checking_Account; Senior_Citizen ISA Person (age ≥ 65).
- **User-defined:** membership assigned by users (no predicate).
Examples: Employee and Team.

Type 2: Disjointness (Can an entity be in multiple subclasses?)

- **Disjoint:** entity can belong to only one subclass.
- **Overlapping:** entity may belong to more than one subclass.

Type 3: Completeness (Must superclass members be in some subclass?)

- **Total specialization/generalization:** every superclass entity belongs to at least one subclass.

- **Partial specialization/generalization:** some superclass entities may belong to no subclass.

Note

Partial is the default. For generalized higher-level sets created by combining subclasses, completeness is often total (e.g., every Student is either Undergrad or Graduate).

Representing Specialization via Schemas

Method 1: One schema for superclass + one schema per subclass

- Superclass schema stores common attributes.
- Each subclass schema stores its local attributes and copies the PK of the superclass as FK (usually also PK).

Example

person (ID, name, street, city)
 student (ID, tot_cred)
 employee (ID, salary)

Drawback: queries about a subclass instance require joining with the superclass.

Method 2: One schema per entity set (subclass duplicates inherited attrs)

- Each subclass schema repeats inherited attributes from the superclass.
- If specialization is *total*, the superclass base table can be omitted (or defined as a *view* that is the union of subclasses).
- Explicit superclass schema may still be useful to host foreign keys.

Example

person (ID, name, street, city)
 student (ID, name, street, city, tot_cred)
 employee (ID, name, street, city, salary)

Drawback: redundant storage for people who are both students and employees.

Aggregation

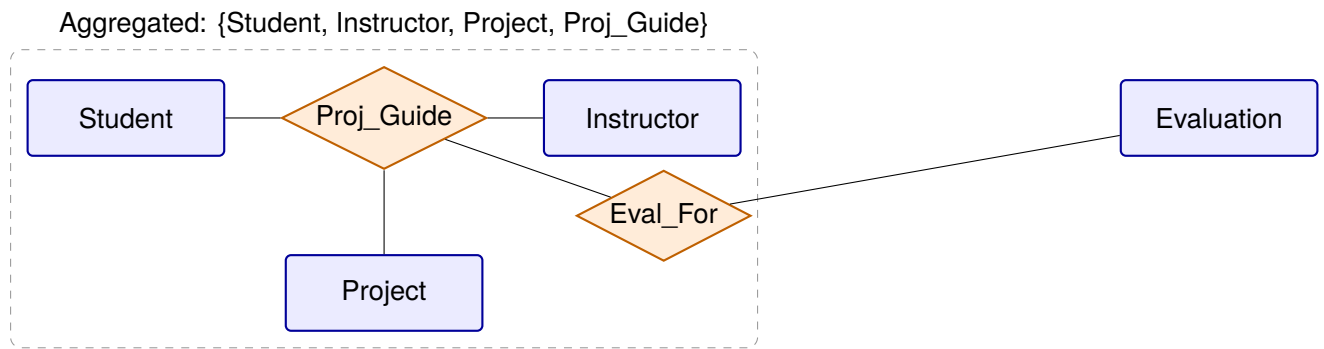
Definition

Aggregation treats a relationship (often with participating entities) as an *abstract entity* so that we can relate it to other entities/relationships without redundancy. Useful when a relationship (e.g., Proj_Guide(Student, Instructor, Project)) itself must participate in another relationship (e.g., Eval_For).

Example

We want to record evaluations of a student by a guide on a project.

Idea: Group Student, Instructor, Project, and Proj_Guide into an aggregate and connect it to Evaluation via Eval_For.



Schemas Corresponding to Aggregation

- Create a schema that contains:
 - The primary key of the aggregated relationship.
 - The primary key of the associated entity set.
 - Any descriptive attributes.

Example

Example formulation: If we aggregate Proj_Guide and relate it to Evaluation via Eval_For, one possible schema is:

eval_for(s_ID, project_ID, i_ID, evaluation_ID)

Note

If we are willing to allow NULLs for some attributes, the original relationship (e.g., Proj_Guide) may become redundant; otherwise, we keep it. Aggregation helps avoid overlapping information between relationships (e.g., Eval_For vs Proj_Guide).

E–R Design Decisions (Checklist)

- Represent an object as an attribute or as an entity?

- Use an entity set or a relationship set for a concept?
 - Prefer a ternary relationship or a pair of binary relationships?
 - Strong vs weak entity set?
 - Use specialization/generalization to improve modularity?
 - Use aggregation to treat complex relationship structures as a single unit?
-